

POLITECNICO
MILANO 1863

054323 - INTERNET OF THINGS

POLITECNICO DI MILANO

COMPUTER SCIENCE AND ENGINEERING

IoT Challenge 3
Part 1

Authors:
Carmen Giaccotto (Person Code: ********)
Alessia Franchetti-Rosada (Person Code: ********)

Date: May 12, 2026

1 Node-RED Implementation

This section details the logic and implementation of the Node-RED flow designed to process the `challenge3.csv` dataset, filter ZigBee packets, extract relevant data and publish information to both a local MQTT broker and a public ThingSpeak channel.

1.1 Overall Flow Architecture

The system is designed to operate by first loading the provided dataset into global memory to optimize read operations. It then generates IoT telemetry data, subscribes to its own MQTT topic to fetch specific rows from the dataset and uses a Switch node to route packets based on their command strings (either `ZBEE_ZCL` or `Link Status`).

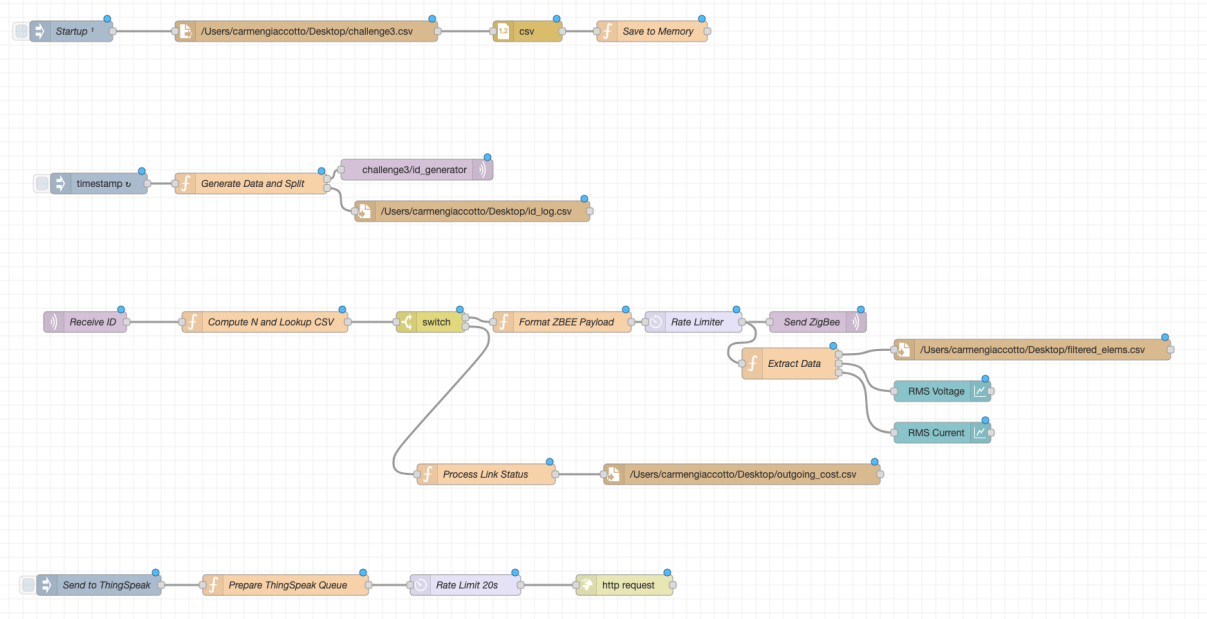


Figure 1: Complete Node-RED Flow Diagram.

1.2 Node Summary

Table 1 lists every node in the flow with its type and purpose.

Node Name	Type	Purpose
Startup	Inject	Fires once on deployment to trigger CSV loading
File In	File In	Reads `challenge3.csv` from disk as a UTF-8 string
CSV	CSV	Parses the string into a JavaScript array of row objects
Save to Memory	Function	Stores the array in `global.csvData` for fast lookup
Inject (1s)	Inject	Clock node firing every 1 second
Generate Data and Split	Function	Generates random ID and UNIX timestamp; routes to MQTT and CSV file
MQTT Out (publisher)	MQTT Out	Publishes `{id, timestamp}` to `challenge3/id_generator` on port 1884
File (id_log)	File	Appends each row to `id_log.csv`; stops after 200 rows
Receive ID	MQTT In	Subscribes to `challenge3/id_generator`
Compute N and Lookup CSV	Function	Counts messages (hard stop at 200); computes $N = ID \bmod 5218$; retrieves matching CSV row
Switch	Switch	Routes message to ZBEE_ZCL branch or Link Status branch based on Command String
Format ZBEE Payload	Function	Builds JSON payload with hex-padded addresses and dynamic MQTT topic
Rate Limiter	Delay	Limits ZBEE publishes to 10 messages per minute (queue mode, no drop)
Send ZigBee	MQTT Out	Publishes the ZBEE payload to topic `ZigBee/<DeviceName>`
Extract Data	Function	Extracts RMS/Power attributes by position; skips incomplete fields; writes `filtered_elems.csv`

Continued on the following page

Table 1 - Continued from previous page

Node Name	Type	Purpose
File (filtered_elems)	File	Appends filtered attribute rows to `filtered_elems.csv`
RMS Voltage Chart	UI Chart	Displays RMS Voltage values on the Node-RED Dashboard
RMS Current Chart	UI Chart	Displays RMS Current values on the Node-RED Dashboard
Process Link Status	Function	Parses Link Status packet; updates global cost map `linkCosts[src][dst]`
File (outgoing_cost)	File	Overwrites `outgoing_cost.csv` with the updated cost map
Send to ThingSpeak	Inject	Manual trigger to start ThingSpeak upload after 200 messages
Prepare ThingSpeak Queue	Function	Finds smallest hex source; sorts destinations; builds message queue
Rate Limit 20s	Delay	Limits ThingSpeak updates to 1 message every 20 seconds (queue mode)
HTTP Request	HTTP Request	Sends GET request to ThingSpeak API with `field1=cost`

Table 1: Summary of all Node-RED nodes and their functions.

1.3 Node Explanation

The architecture is divided into five logical blocks to address all the challenge requirements.

1.3.1 Block 0: Memory Initialization

Upon deployment, an **Inject** node fires once automatically (after a 0.1 s delay). It triggers a **File In** node that reads `challenge3.csv` as a UTF-8 string. A **CSV** node then parses the content into a JavaScript array of objects, one per row. A final **Function** node ("Save to Memory") stores this array into a global variable:

```
global.set("csvData", msg.payload);
```

This makes the dataset instantly accessible to the rest of the flow without repeated disk reads.

1.3.2 Block 1: ID Generator and Logger

An **Inject** node acts as a clock, firing every 1 second. It connects to a **Function** node ("Generate Data and Split") that:

- Generates a random integer ID between 0 and 30000.
- Records the current UNIX timestamp via `Math.floor(Date.now() / 1000)`.
- Tracks the row number using a `context` variable.

The node outputs through two separate pins:

- **Pin 1:** Formats a JSON payload `{"id": ..., "timestamp": ...}` and sends it to an **MQTT Out** node, publishing to topic `challenge3/id_generator` on `localhost:1884`.
- **Pin 2:** Formats a CSV row and passes it to a **File** node that appends it to `id_log.csv`. The header `No.,ID,TIMESTAMP` is written only on the first row. Writing stops after 200 rows.

1.3.3 Block 2: Subscription and Data Retrieval

An **MQTT In** node subscribes to `challenge3/id_generator`. A **Function** node ("Compute N and Lookup CSV") implements a hard counter using context variables: after exactly 200 messages, all subsequent ones are silently dropped. For each valid message, it computes:

$$N = ID \bmod 5218 \tag{1}$$

It then searches the globally stored `csvData` array to find the row whose `Packet Number` equals N. The row and the original ID are attached to the message and forwarded.

1.3.4 Block 3: Packet Routing

A **Switch** node evaluates the `Command String` field of the retrieved row with two rules ("check all" mode enabled):

- **Output 1:** Routes to the ZBEE_ZCL branch if the string contains `ZBEE_ZCL`.
- **Output 2:** Routes to the Link Status branch if the string contains `Link Status`.

Messages matching neither rule are silently dropped but still count toward the 200-message limit.

1.3.5 Block 4: ZBEE_ZCL Processing, Attribute Extraction and Dashboard

For ZBEE_ZCL packets, a **Function** node ("Format ZBEE Payload") builds the required JSON payload, converting all addresses to hexadecimal format using a `toHex()` helper with 4-digit zero-padding. The device name has + characters removed to comply with MQTT topic naming rules. The message topic is set dynamically to `ZigBee/<DeviceName>`. The message then passes through a **Delay** node configured as a rate limiter (10 messages per minute, queue mode - no messages are dropped) before reaching an **MQTT Out** node.
After the rate limiter, a **Function** node ("Extract Data") checks whether the Command String contains `Active Power`, `RMS Current` or `RMS Voltage`. If so, it uses regular expressions to extract the Sequence Number, Attribute list, Status list, Data Type list and numeric values from `Int16/Uint16` fields. Attributes are matched to their status, data type and value by position index. Attributes missing a Status or Data Type field are excluded per the professor's specification.
Matching rows are appended to `filtered_elems.csv`. Numeric values are also forwarded to two **UI Chart** nodes (RMS Voltage and RMS Current) displayed on the Node-RED Dashboard under the group "Sensori".

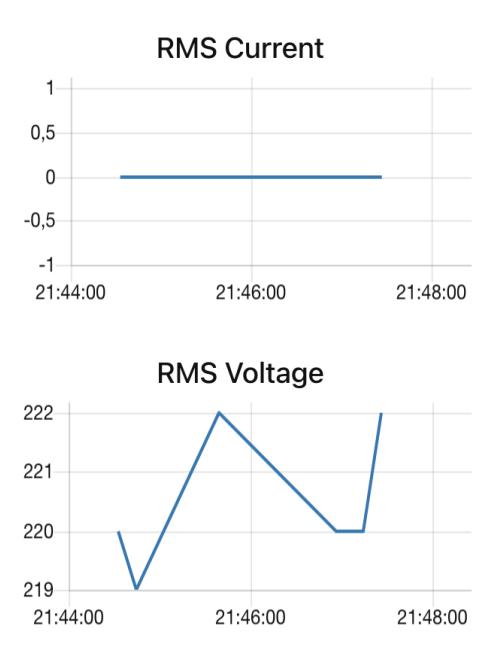


Figure 2: Node-RED Dashboard displaying RMS Current and RMS Voltage plots.

1.3.6 Block 5: Link Status Processing and Cloud Integration

For Link Status packets, a **Function** node ("Process Link Status") parses the Command String by normalizing Python-style single quotes to JSON double quotes, then calling

`JSON.parse()`. It extracts the `Links` array and updates a global cost map:

```
linkCosts[source][destination] = outgoing_cost;
```

If a later packet from the same source arrives, the stored costs are updated (overwritten). After each update, the full `outgoing_cost.csv` is rewritten from the current in-memory map.
A manual **Inject** node ("Send to ThingSpeak") triggers a **Function** node ("Prepare ThingSpeak Queue") that:

1. Finds the smallest source address treating all values as hexadecimal integers.
2. Sorts its destination addresses in ascending hexadecimal order.
3. Returns an array of `{payload: cost}` messages.

These pass through a **Delay** node (rate limit: 1 message every 20 seconds, queue mode) to an **HTTP Request** node that sends a GET request to:

```
https://api.thingspeak.com/update?api_key=******** &field1
={{{payload}}}
```

2 Results

2.1 id_log.csv

The flow generated and logged exactly 200 messages. The file has the following format:

```
No., ID, TIMESTAMP
1,28804,1778550093
2,22080,1778550094
...
200,28053,1778550292
```

2.2 filtered_elems.csv

30 rows were extracted across 10 qualifying packets (3 attributes each: Active Power, RMS Current, RMS Voltage). All rows include Status and Data Type fields.
Sample rows:

```
No., Timestamp, Sequence Number, Attribute, Status, Data Type, Data Value
1,1778550140,11,Active Power,Success,16-Bit Signed Integer,0
2,1778550140,11,RMS Current,Success,16-Bit Unsigned Integer,0
3,1778550140,11,RMS Voltage,Success,16-Bit Unsigned Integer,220
4,1778550145,11,Active Power,Success,16-Bit Signed Integer,0
```

```
5,1778550145,11,RMS Current,Success,16-Bit Unsigned Integer,0
6,1778550145,11,RMS Voltage,Success,16-Bit Unsigned Integer,219
```

2.3 outgoing_cost.csv

70 rows were recorded across 9 unique source addresses. The smallest source address in hexadecimal is `0x0000`. Its destination addresses and outgoing costs, sorted in ascending hexadecimal order, are shown in Table 2.

Destination	Outgoing Cost
0x0112	7
0x1cd8	1
0x1e15	5
0x3d95	3
0x4e11	3
0xa706	3
0xe1a6	1
0xec7f	1

Table 2: Outgoing costs from source `0x0000`, sorted by destination address.

3 Cloud Integration

The 8 outgoing cost values for source `0x0000` were successfully transmitted to a public ThingSpeak channel in destination-address order, respecting the 20-second rate limit between messages.

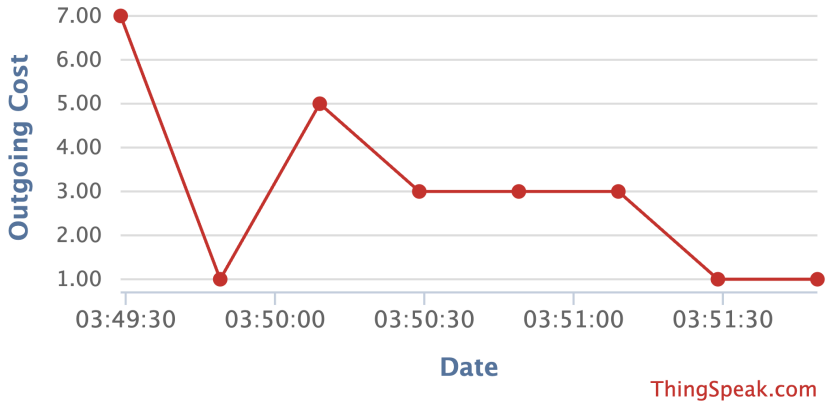


Figure 3: ThingSpeak channel showing the uploaded outgoing costs for source `0x0000`.

ThingSpeak Channel ID: ********